Building AI Agents for Finance: Reading Notes for Ambition

Nov 3, 2026 · Dayn Kirksey

Building AI Agents for Finance by Hanane Dupouy and Fayssal El Mofatiche (Packt, August 2026, 758 pages) shapes how Ambition's agents spend safely, but it never mentions blockchains, wallets, stablecoins or agent payments. Its central rule is that the model is one component and the system around it is the product, which is exactly Ambition's bet. Page numbers below are the book's own.

The book at a glance

Chapter	Pages	Core idea
1. What are AI agents?	1–43	Start with the simplest solution that works, and add autonomy only when it fails
2. Design patterns	45–108	Agents are composed from capability, reasoning, quality and orchestration patterns, with memory and context engineering as the hard parts
3. Frameworks	109–167	No framework is best; score them on nine dimensions, with auditability and safety weighted highest in finance
4. Fundamental analysis system	169–243	Push deterministic logic into tools and let the model reason over verified inputs
5. Deep search	245–289	Code-based checks of numbers beat model-based checks; financial data is "adversarial"
6. Reasoning paradigms	291–369	Match the reasoning method (ReAct, reflection, voting, tree search) to the shape of the problem
7. Multi-agent systems	371–425	Seven architectural styles; add agents only when measurable value justifies them
8. Trading committee and debate	427–479	Committees drift toward consensus, so add a bull, a bear, a devil's advocate and a judge on different models
9. Insurance workflows	481–515	Fraud review as a debate between a fraud advocate and a legitimacy advocate; the audit trail is the product
10. Agentic RAG	517–565	Retrieval that routes, decomposes and checks itself, with citations for every claim
11. Evaluating agents	567–617	Six quality dimensions, three test layers, severity-driven scorecards and model risk reports
12. Operating agents responsibly	619–689	The harness, budgets, guardrails, versioning, autonomy tiers and a readiness checklist

Rules we adopt

Twelve rules from the book change what we build, and most of them come from chapter 12, because Ambition is mainly the safety machinery around an agent that spends money.

Rule	Pages	In Ambition
Retry reads freely, but never blindly retry an action; protect actions with idempotency keys and, usually, human approval	625–641	Balance and price reads retry with backoff; every payment carries an idempotency key, and `SpendConfirmation`'s double-send guard is the human layer
Enforce permissions in the harness, not the prompt	625–641	The spending limit lives in the contract and the recipient allow-list in the tool layer, so a fooled model still can't overspend
Secrets never reach the model	625–641	Wallet keys are injected at signing time and never appear in prompts, tool descriptions or context
Budgets are risk limits, and "insufficient evidence" is a valid answer	641–647	Every agent run has turn and token budgets plus a repeat-call detector, alongside the money limit
Assign every action an autonomy tier: inform, propose, act with approval, or act and report	680–685	The book reserves act-and-report for low-risk, reversible actions, and payments are final, so Ambition makes them low-risk instead: small amounts, a contract-enforced daily limit and money-back escrow. Anything above the limit or to a new recipient moves up to act-with-approval
Guard three points (input, tool output, final output) with allow, block, redact or escalate, running deterministic checks first	666–680	x402 payment requests and vendor responses are tool outputs: treat them as data, and check the price against the limit in code before any model sees it
One audit record binds the request, evidence, trace, version, guardrail decisions, conclusion and approver	509–512, 680–685	The statements module is that record, anchored by transaction hashes
A version pins the model, prompts, tools and configuration, and a prompt change is a code change	661–666	Pin exact model IDs, and hash prompts into a manifest that tags each release
Do deterministic work in code, and let the model reason over verified inputs	239–243, 272–277	Amounts are strings or big integers, computed in code, never by the model
Evaluate on six dimensions and rank failures by severity, not by weighting alone	572–617	Overspending, a wrong recipient or a leaked key are catastrophic and need 100% pass rates; an empty read reported as "no spending" is a trajectory failure, which `MoneyEmptyState` prevents
Argue both sides before acting: a fraud advocate and a legitimacy advocate on different, equally strong models, each naming its own weaknesses	466–479, 505–509	`AgentProposalLabel` can show the agent's stated reason and its weakest point
Start with one agent and good tools, and add agents only for measurable value	36–37, 412–417	One agent with tools carries weeks 1 to 12; a second agent appears only if evaluation shows it helps

What the book doesn't cover

The book never mentions blockchains, smart contracts, wallets, stablecoins, x402 or agent payments, so it informs the agent and its safeguards but not the on-chain parts. Contract security and gas costs still come from the plan and the code. Its labs are in Python with OpenAI, Claude, LangGraph and LlamaIndex, and its framework survey is current to mid-2026, so check framework details before relying on them. Treat its customer statistics as vendor claims, as the book itself often notes.

Companion notebooks: Dupouy's LLMs-in-Finance

Hanane Dupouy's own notebook collection, hananedupouy/LLMs-in-Finance, turns several of the book's evaluation and cost rules into runnable code under the MIT license. We link to it rather than copy it in, and every link below is pinned to commit `d6d4d4e` (November 1, 2026), the version reviewed, so the files can't change underneath us. The repo is for education rather than production, and like the book it never covers blockchains or payments.

Notebook	What it shows	In Ambition
Trajectory evals with Phoenix	An agent gives the right number without calling the approved calculator, and a check on the path it took, rather than the answer, catches it; the tracing runs on your own machine	Prove every spend passed the limit check and every amount came from code
Prompt caching with Anthropic and with OpenAI in a RAG workflow	Reusing a cached prompt prefix, so agents that re-read the same material cost less and answer faster	Cheaper runs when agents re-read the same docs or statements
RAG evaluation with DeepEval and Giskard	Scoring answers drawn from looked-up documents	A starting scorecard for any feature that answers from documents
Paper summaries	Short notes on evaluation-driven development, chain-of-verification, self-refine and StockBench	A reading list for the evaluation work

Check Arize Phoenix's own license before adopting it. The Giskard notebook, like several others in the repo, sends documents to the LlamaParse cloud service, which is fine for public material only. Copying any code means keeping its MIT notice and naming the source and commit in the commit message.

How the book is stored and used

The PDF itself stays out of Git. The book's copyright page says no part may be "reproduced, stored in a retrieval system, or transmitted" without Packt's permission, apart from brief quotations, and a Git repository is a retrieval system. This repo is public and mirrored to GitLab, so the book can never be committed here.

These notes are what Git stores. They summarize the book in my own words, with page numbers and only short quotations, which is how plans, code reviews and refactors should reference it. They live in this repo and mirror to GitLab with it.

Keep your copy where you can hand it to me. Store the PDF in a folder on your computer or in your own cloud storage. This workspace is cleared between sessions, so when a task needs the book's detail, attach it again, or link the chat to your computer and I'll read it from that folder.

Use the companion code by link. The book's code lives in Packt's public repository, PacktPublishing/Building-AI-Agents-for-Finance. Check its license before copying any of it, then keep its notice and name the source in the commit message.

Cite the book where its rules land. Code comments, design decisions and pull requests can cite a rule as "Dupouy and El Mofatiche, p. 641", so the reason behind a safeguard survives every refactor.